

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Distributed Computing

Michael Kosyakov
Associate Professor

Denis Tarakanov
Assistant Lecturer

ifmo.distributedclass.bot@gmail.com

Communication primitives

Send and receive primitives



Send and receive are two basic operations
The prototypes of these operations are as follows:

send(void *sendbuf, int count, int dest)

- sendbuf – buffer in user space containing data to be sent
- count – number of bytes to send
- dest – identifies the destination process

receive(void *recvbuf, int count, int source)

- recvbuf – user buffer into which data is to be received
- count – number of bytes to receive
- source – identifies the source from which data is to be received

Ananth Grama, Anshul Gupta, George Karypis, Vipin Kumar. Introduction to Parallel Computing, Second Edition, Pearson, 2003

Send and receive primitives



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Consider the following code:

P_1	P_2

a = 100;
send(&a, 1, 2);
a = 0;
	receive(&a, 1, 1);
	printf("%d\n", a);

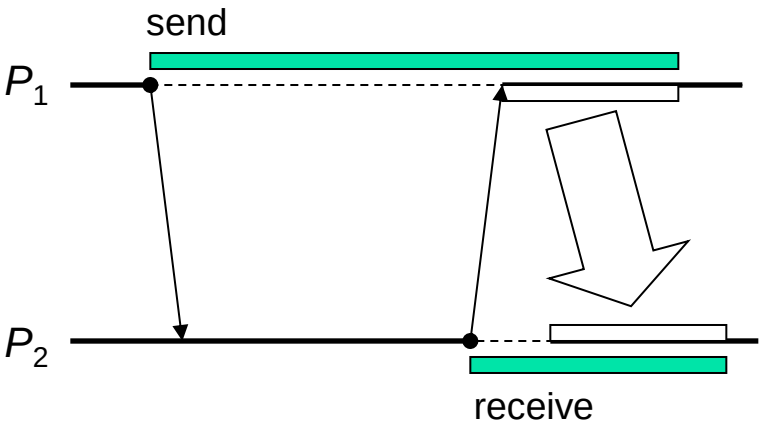
- Semantics of send operation requires that P2 receives value 100 and not 0
- This motivates the design of the send and receive protocols

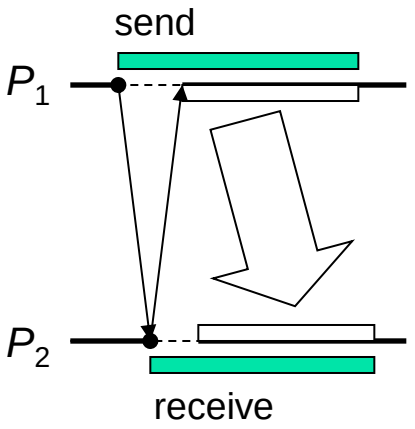
Blocking Operations

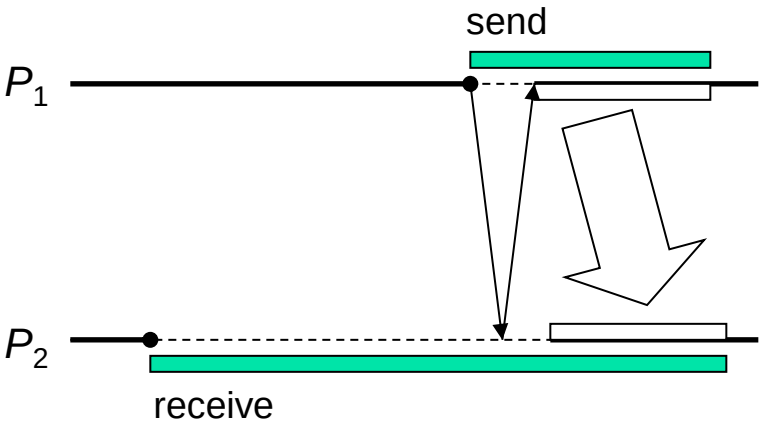


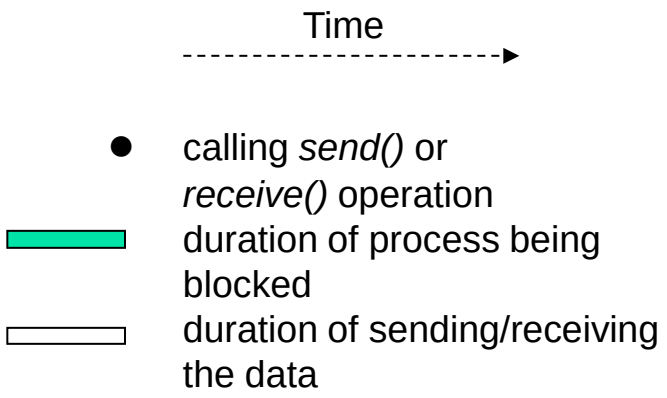
- Send/receive semantics can be ensured if send operation blocks until it is safe to return
  - In general, it does not mean that the receiver has received the data
- Blocking Non-Buffered Send/Receive
  - No buffers are used at either sending or receiving side
  - Send operation does not return until the matching receive has been invoked and the receive has been completed
  - Receive operation completes when the data is copied into the receiver's user buffer
  - Typically, send and receive handshake with each other

Blocking Non-Buffered Send/Receive









Blocking Non-Buffered Send/Receive



The following code leads to a deadlock:

P_1	P_2
send(&a, 1, 2); receive(&b, 1, 2);	send(&a, 1, 1); receive(&b, 1, 1);

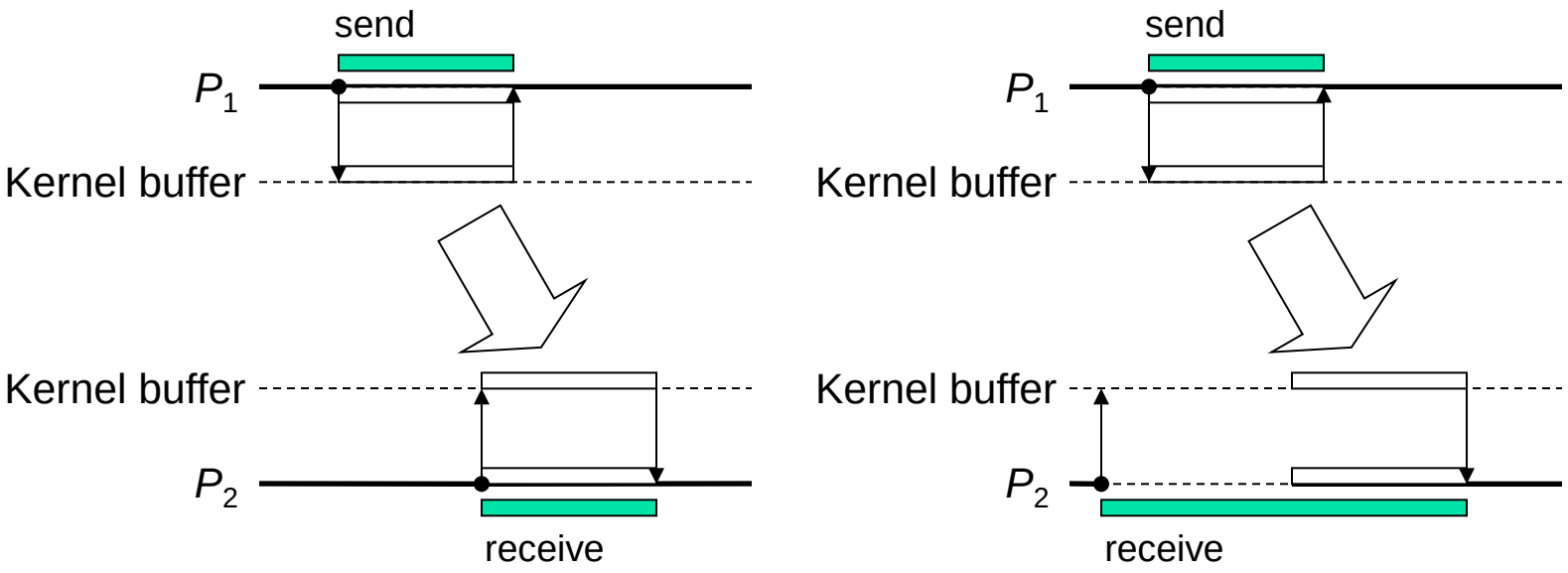
- Deadlock could be resolved by swapping operations in P1 or P2, but this makes code more cumbersome
- Idling and deadlocks are major drawbacks of non-buffered blocking sends

Blocking Buffered Send/Receive



- A simple solution to the problems outlined above is to rely on buffers at the sending and receiving sides
- Sender copies the data to the kernel buffer and returns after copying has been completed
  - Any further changes to the data will not impact program semantics
- At the receiving side the data cannot be stored directly at the target location
  - The data must be copied at the kernel buffer as well
- Buffering mitigates idling at the cost of adding buffer management overheads

Blocking Buffered Send/Receive



- calling *send()* or *receive()* operation
- duration of process being blocked
- duration of copying the data to/from kernel buffer

Blocking Buffered Send/Receive



Impact of finite buffers on performance:

P_1	P_2

for (i = 0; i < 1000; i++) {
produce_data(&a);
send(&a, 1, 2);
}
	for (i = 0; i < 1000; i++) {
	receive(&a, 1, 1);
	consume_data(&a);
	}

- What if consumer is much slower than producer?
- In case of buffer overflow the sender will block …

Blocking Buffered Send/Receive



Deadlocks are still possible with buffering:

P_1	P_2
receive(&b, 1, 2); send(&a, 1, 2);	receive(&b, 1, 1); send(&a, 1, 1);

- Both processes wait to receive data but neither one sends it
- Receive operation is still blocking to ensure program semantics

Blocking Buffered Send/Receive



Can deadlock happen if both processes try to send?

P_1	P_2
send(&a, 1, 2); receive(&b, 1, 2);	send(&a, 1, 1); receive(&b, 1, 1);

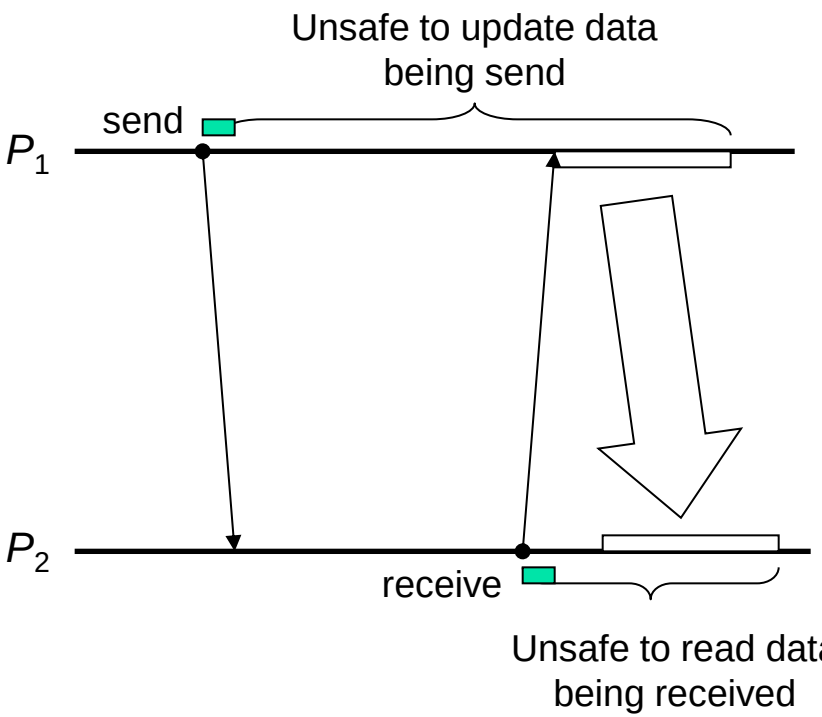
- If all of the buffer space at P1 is occupied by data destined to P2, and reverse is true for P2, then neither side can send any more data

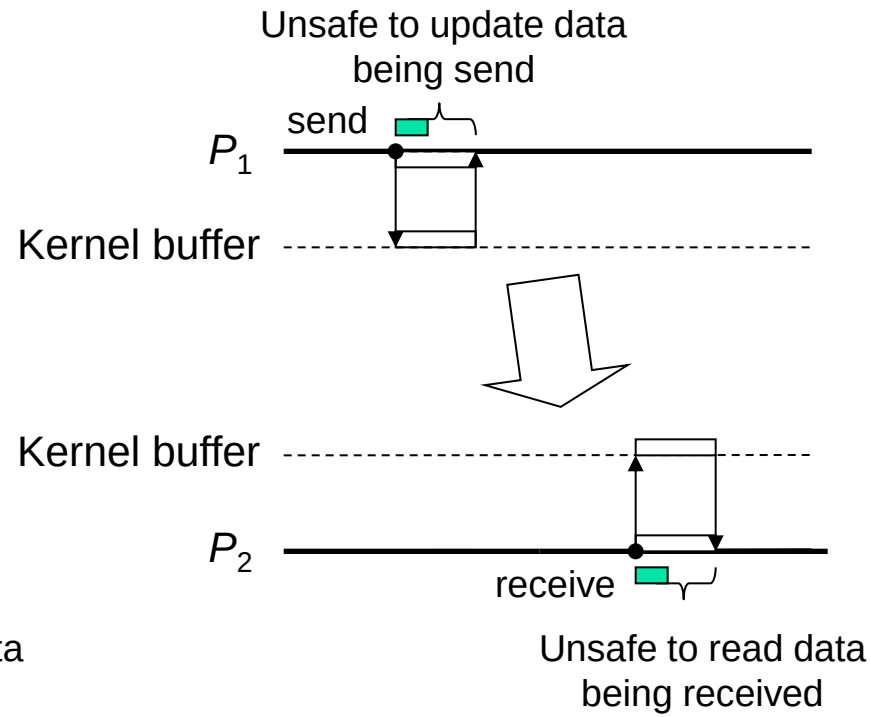
Non-Blocking Operations



- Non-blocking operations return immediately after invocation even though the operation has not completed
  - Send operation returns even before the data is copied out of the user buffer
  - Receive operation returns even before the data may have arrived from the sender
- Programmer must not alter the data that may be potentially participating in communication
- Non-blocking send/receive are generally accompanied with check-for-completion operations

Non-Blocking Operations





Non-Blocking Operations



- Non-blocking operations mitigate idling at the cost of some program restructuring
- When used correctly, communication overhead can be almost entirely masked, i.e. communication can be overlapped with useful computations
- Non-blocking primitives can’t cause deadlock
- Message passing libraries typically provide both blocking and non-blocking primitives

Synchronous vs Async. primitives



- Blocking / non-blocking categories describe operations behavior from the *local* view of the invoking process, without taking other processes into account
  - Either return when the operation “semantically completes”
  - Or just initiate the operation
- But it is also useful to consider the effect of communication operations from a *global* viewpoint
- Hence, it is reasonable to distinguish between synchronous and asynchronous communications

Synchronous Communication



- Send completes only after the invoking process learns that matching Receive has also been invoked and that the receive operation has been completed
  - Receive should reply with acceptance
  - In general, no need in kernel buffers
- Receive completes when the data to be received is copied into the receiver's user buffer
- Hence, there is a form of synchronization between the sending and the receiving processes
- But ... deadlock is possible (much like with blocking non-buffered operations)

Asynchronous Communication



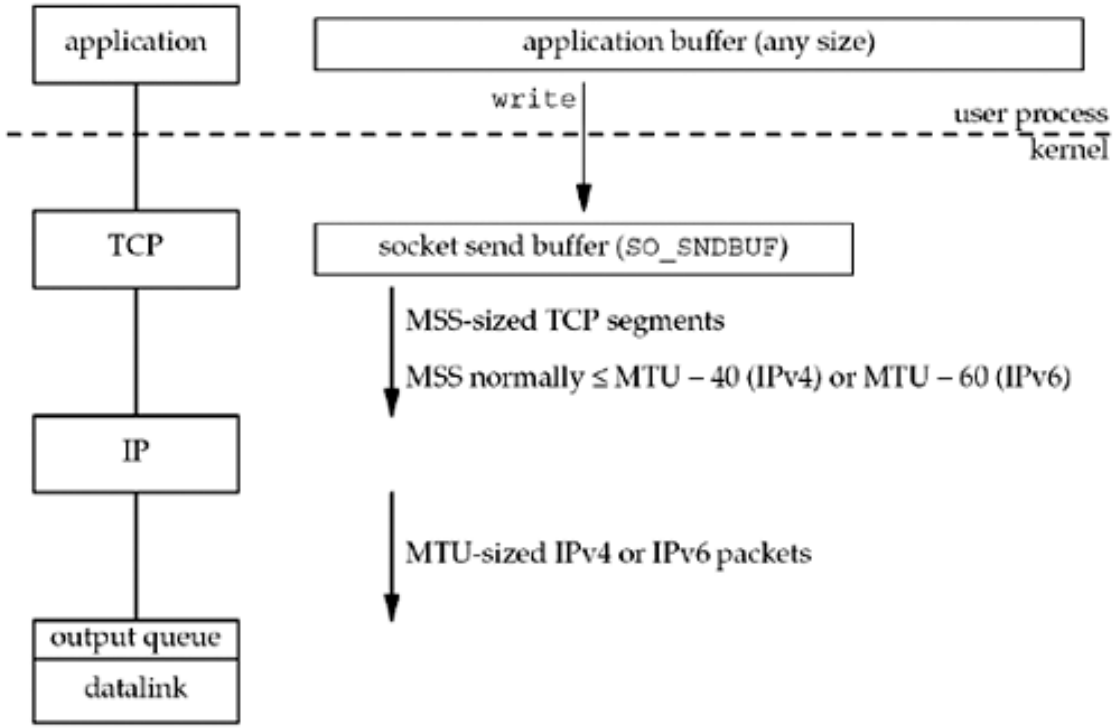
- Sender can execute its communication operation without any coordination with the receiving process
  - Sending process does not wait for the receiving process to receive the message
  - Hence, sender does not know if its previous message was received or not (in contrast to blocking synchronous send)
  - The receiving process does not issue an acceptance
  - Communication requires buffering for the messages that have been sent but not received
- It does not make sense to define asynchronous receive primitive

TCP: Asynchronous Blocking Buffered



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Steps and buffers involved when an application writes to a TCP socket



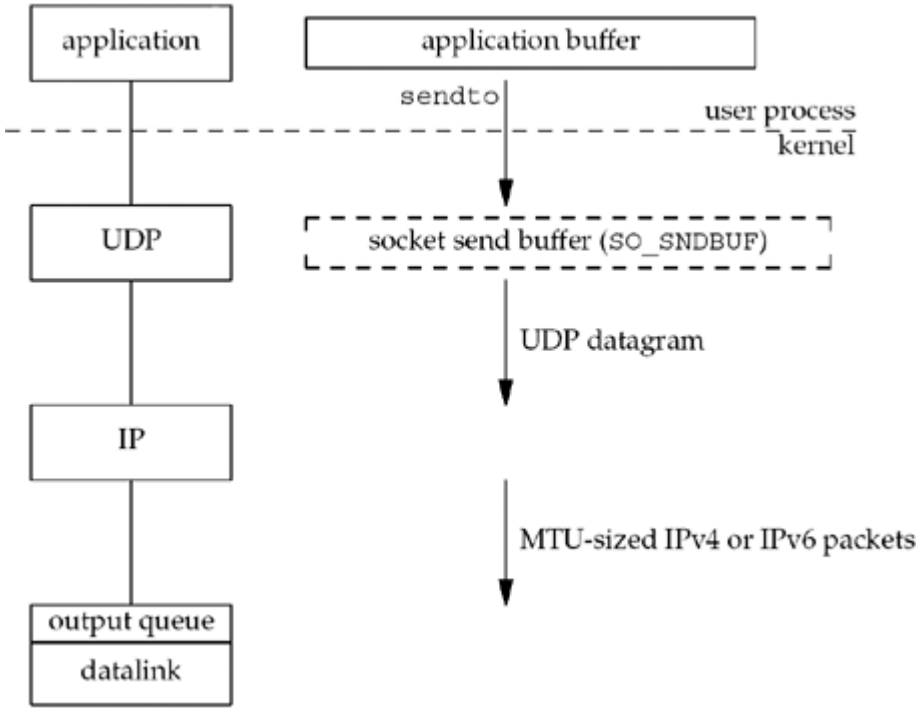
Successful return from a `write` to a TCP socket only tells us that we can reuse our application buffer. It does *not* tell us that either the peer TCP has received the data or that the peer application has received the data.

UDP: Asynchronous Blocking Buffered



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Steps and buffers involved when an application writes to a UDP socket



Successful return from a `write` to a UDP socket tells us that either the datagram or all fragments of the datagram have been added to the datalink output queue. If there is no room on the queue for the datagram or one of its fragments, ENOBUFS is often returned to the application.

TCP vs UDP

- TCP takes the data in the socket send buffer and sends it to the peer TCP based on all the rules of TCP data transmission. The peer TCP must acknowledge the data, and as the ACKs arrive from the peer, only then can our TCP discard the acknowledged data from the socket send buffer. TCP must keep a copy of our data until it is acknowledged by the peer.
- Since UDP is unreliable, it does not need to keep a copy of the application's data and does not need an actual send buffer. (The application data is normally copied into a kernel buffer of some form as it passes down the protocol stack, but this copy is discarded by the datalink layer after the data is transmitted.)

W. Richard Stevens et al., UNIX Network Programming Volume 1, Third Edition: The Sockets Networking API, 2003

TCP vs UDP

- TCP provides flow control. TCP always tells its peer exactly how many bytes of data it is willing to accept (advertised window). At any time, the window is the amount of room currently available in the receive buffer, so that the sender cannot overflow the receive buffer. The window changes dynamically over time: As data is received from the sender, the window size decreases, but as the receiving application reads data from the buffer, the window size increases. It is possible for the window to reach 0: when TCP's receive buffer for a socket is full and it must wait for the application to read data from the buffer before it can take any more data from the peer.
- UDP provides no flow control. It is easy for a fast UDP sender to transmit datagrams at a rate that the UDP receiver cannot keep up with.

W. Richard Stevens et al., UNIX Network Programming Volume 1, Third Edition: The Sockets Networking API, 2003